

EXPERIMENT 5

Title

Create SQL views for department salary summary and employee hierarchy, test updatability of views, and implement a recursive Common Table Expression (CTE) to display reporting chains.

Aim

To create and use SQL views for department salary information and employee hierarchy, test view updatability, and implement a recursive CTE to display employee reporting chains.

Theory

A view is a virtual table based on a SELECT statement. It provides a reusable way to present data without repeatedly writing the complete query. Views are useful for simplifying complex queries, data abstraction, and controlled access to selected data.

- Simple views based mainly on one table may be updatable when they satisfy MySQL's rules.
- Views containing GROUP BY and aggregate functions such as SUM(), AVG(), or COUNT() are generally not directly updatable.
- A hierarchy view can represent employee-manager relationships using a self-join.
- A recursive CTE consists of an anchor query and a recursive query and is useful for hierarchical data.
- WITH RECURSIVE is used in MySQL 8.0 and later for recursive CTEs.
- The reporting chain follows manager_id from a manager to direct and indirect reports.

Database Schema Used

```
CREATE TABLE Department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50) UNIQUE NOT NULL,  
    location VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(60) NOT NULL,  
    job_title VARCHAR(50),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    department_id INT,  
    manager_id INT NULL,  
    FOREIGN KEY (department_id) REFERENCES Department(department_id),  
    FOREIGN KEY (manager_id) REFERENCES Employee(employee_id)  
);
```

Add Manager Relationship

```
ALTER TABLE Employee
ADD COLUMN manager_id INT NULL;

ALTER TABLE Employee
ADD CONSTRAINT fk_employee_manager
FOREIGN KEY (manager_id) REFERENCES Employee(employee_id);
```

Sample Employee Hierarchy

```
UPDATE Employee SET manager_id = NULL WHERE employee_id = 1008;

UPDATE Employee SET manager_id = 1008 WHERE employee_id IN (1007,1009);
UPDATE Employee SET manager_id = 1007 WHERE employee_id IN (1011,1012);
UPDATE Employee SET manager_id = 1009 WHERE employee_id = 1010;

UPDATE Employee SET manager_id = 1015 WHERE employee_id IN (1013,1014,1016);
UPDATE Employee SET manager_id = 1014 WHERE employee_id IN (1017,1018);

UPDATE Employee SET manager_id = 1020 WHERE employee_id IN (1019,1021,1022);
UPDATE Employee SET manager_id = 1022 WHERE employee_id IN (1023,1024);

UPDATE Employee SET manager_id = 1025 WHERE employee_id IN (1026,1027,1028);
UPDATE Employee SET manager_id = 1027 WHERE employee_id IN (1029,1030);
```

Department Salary Summary View

Create the View

```
CREATE VIEW department_salary_summary AS
SELECT d.department_id,
       d.department_name,
       COUNT(e.employee_id) AS employee_count,
       SUM(e.salary) AS total_salary,
       AVG(e.salary) AS average_salary,
       MIN(e.salary) AS minimum_salary,
       MAX(e.salary) AS maximum_salary
FROM Department d
LEFT JOIN Employee e
  ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Display the View

```
SELECT * FROM department_salary_summary;
```

Employee Hierarchy View

Create the View

```
CREATE VIEW employee_hierarchy AS
SELECT e.employee_id,
       e.employee_name,
       e.job_title,
       e.department_id,
       e.manager_id,
       m.employee_name AS manager_name
FROM Employee e
```

```
LEFT JOIN Employee m
ON e.manager_id = m.employee_id;
```

Display the View

```
SELECT *
FROM employee_hierarchy
ORDER BY manager_id, employee_id;
```

Test View Updatability

Simple Updatable View

```
CREATE VIEW employee_basic AS
SELECT employee_id, employee_name, job_title, salary, department_id
FROM Employee;
```

Try an update through the view:

```
UPDATE employee_basic
SET salary = salary + 2000
WHERE employee_id = 1001;
```

Verify the change in the base table:

```
SELECT employee_id, employee_name, salary
FROM Employee
WHERE employee_id = 1001;
```

Aggregate View Updatability Test

Try to update the calculated average salary in the aggregate view:

```
UPDATE department_salary_summary
SET average_salary = 90000
WHERE department_id = 1;
```

Hierarchy View

For hierarchy changes, update the Employee base table directly.

```
UPDATE Employee
SET manager_id = 1008
WHERE employee_id = 1007;
```

Recursive CTE for Reporting Chains

Purpose

The recursive CTE starts at a top-level employee and repeatedly finds employees whose manager_id matches the current employee_id.

Query

```
WITH RECURSIVE reporting_chain AS (
    SELECT employee_id,
           employee_name,
           manager_id,
           0 AS level,
           CAST(employee_name AS CHAR(1000)) AS reporting_path
    FROM Employee
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.employee_id,
           e.employee_name,
           e.manager_id,
           r.level + 1,
           r.reporting_path || ',' || e.employee_name
    FROM Employee e
    JOIN reporting_chain r
    ON e.manager_id = r.employee_id;
```

```

        SELECT e.employee_id,
               e.employee_name,
               e.manager_id,
               rc.level + 1,
               CONCAT(rc.reporting_path, ' -> ', e.employee_name)
FROM Employee e
INNER JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id
)
SELECT employee_id,
       employee_name,
       manager_id,
       level,
       reporting_path
FROM reporting_chain
ORDER BY reporting_path;

```

Recursive CTE for a Specific Manager

```

WITH RECURSIVE reporting_chain AS (
    SELECT employee_id,
           employee_name,
           manager_id,
           0 AS level,
           CAST(employee_name AS CHAR(1000)) AS reporting_path
FROM Employee
WHERE employee_id = 1008

    UNION ALL

    SELECT e.employee_id,
           e.employee_name,
           e.manager_id,
           rc.level + 1,
           CONCAT(rc.reporting_path, ' -> ', e.employee_name)
FROM Employee e
INNER JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id
)
SELECT *
FROM reporting_chain
ORDER BY level, employee_id;

```

Display Reporting Chain with Indentation

```

WITH RECURSIVE reporting_chain AS (
    SELECT employee_id, employee_name, manager_id, 0 AS level
FROM Employee
WHERE manager_id IS NULL

    UNION ALL

    SELECT e.employee_id, e.employee_name, e.manager_id, rc.level + 1
FROM Employee e
JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id

```

```

)
SELECT employee_id,
       CONCAT(REPEAT(' ', level), employee_name) AS reporting_employee,
       manager_id,
       level
FROM reporting_chain
ORDER BY level, employee_id;

```

Additional Queries on the Views

Departments with Average Salary Above 75000

```

SELECT department_name, employee_count, average_salary
FROM department_salary_summary
WHERE average_salary > 75000
ORDER BY average_salary DESC;

```

Employees and Their Managers

```

SELECT employee_name, job_title, manager_name
FROM employee_hierarchy
ORDER BY manager_name, employee_name;

```

Top-Level Employees

```

SELECT employee_id, employee_name, job_title
FROM employee_hierarchy
WHERE manager_id IS NULL;

```

Number of Direct Reports

```

SELECT m.employee_name AS manager,
       COUNT(e.employee_id) AS direct_reports
FROM Employee m
LEFT JOIN Employee e
  ON e.manager_id = m.employee_id
GROUP BY m.employee_id, m.employee_name
ORDER BY direct_reports DESC;

```

Result

The required SQL views were created for department salary summaries and employee hierarchy. View updatability was tested using simple and aggregate views. A recursive CTE was implemented to display employee reporting chains and hierarchy levels.

Conclusion

SQL views provide reusable and simplified access to database information. Aggregate views are useful for reporting but are generally not directly updatable. Self-join based hierarchy views help display manager relationships, while recursive CTEs are especially useful for traversing employee reporting chains.